



EdgeK BEAST

Governed Hybrid Inference Gateway for Agentic Coding

POWER WITHOUT WASTE

Hybrid Local-Cloud Inference

MCP Tool Governance

Context Economy

Runtime Control

Document type: Integrated architecture whitepaper with visual appendix

Prepared for: Byron Bunt

Updated: 12 June 2026

Status: Concept architecture, integration summary, commercialization-ready discussion draft

| Table of Contents

1. Executive Summary
 2. What EdgeK BEAST Is
 3. Why the Core Idea Matters
 4. Architecture Overview
 5. The Local-First Hybrid Inference Cascade
 6. Major Integrations and Their Roles
 7. Context Interception and Economization
 8. Runtime Governance and Swarm Control
 9. User Interface / Control Plane Vision
 10. Latest Benchmark Review
 11. Patent-Supported Novelty Highlights
 12. Commercial Relevance
 13. MVP, Deployment Tiers, and Roadmap
 14. Practical Risks and Design Boundaries
 15. Conclusion
-

| 1. Executive Summary

EdgeK BEAST is best understood as a **governed AI execution gateway** for agentic coding, not as a simple model router and not as a token-bypass gimmick. It sits between developer-facing AI clients, local inference tools, external model APIs, MCP tool servers, repository intelligence services, and observability systems.

Its purpose is to make AI coding workflows:

- **cheaper**, by reducing redundant context before it reaches expensive model endpoints;
- **safer**, by governing tool access, redacting secrets, and protecting critical files;
- **faster**, by routing routine work to local models and escalating only hard reasoning to cloud APIs;
- **more reliable**, by introducing runtime pacing, queueing, circuit breakers, and validation layers; and
- **more intelligent over time**, by learning reusable tool patterns from successful traces.

The central strategic leap in the BEAST concept is the move from passive proxying to **active context governance**. Instead of allowing coding agents to flood the model with raw files, logs, schema metadata, and tool definitions, BEAST intercepts the context acquisition process, distills the relevant subset, packages it as a structured handoff packet, and then decides whether local inference is enough or whether the task should be escalated to an enterprise API such as NVIDIA NIM, OpenAI, Gemini, or OpenRouter.

That makes BEAST a **hybrid local-cloud inference control plane**.

2. What EdgeK BEAST Is

At its most mature, EdgeK BEAST is a unified system made up of the following functional layers:

1. **Gateway layer** – accepts traffic from Claude Code, Codex CLI, OpenAI-compatible tools, Anthropic-compatible tools, and future developer clients.
2. **Normalization layer** – converts heterogeneous requests into a provider-neutral internal representation.
3. **Context economy layer** – estimates, reduces, retrieves, ranks, compresses, and annotates context.
4. **Tool governance layer** – brokers MCP, shell, database, filesystem, and repository tools under policy.
5. **Inference routing layer** – decides what runs locally and what gets escalated to cloud APIs.
6. **Runtime governor** – applies queueing, stasis-wall pacing, circuit breakers, timeouts, and retry suppression.
7. **Validation and security layer** – verifies patches, blocks dangerous actions, redacts secrets, and detects prompt injection.
8. **Forensic and learning layer** – archives traces, records metrics, builds workspace graphs, and promotes repeated successful workflows into reusable meta-tools.
9. **Control-plane UI** – exposes the whole system through a dashboard with metrics, graphs, alerts, and governance views.

In plain language: **BEAST turns agentic coding from a noisy chain of expensive guesses into a disciplined engineering workflow.**

3. Why the Core Idea Matters

Most AI coding stacks optimize at the wrong level. They focus only on the provider or the model. BEAST focuses on the **workflow before the expensive model call**.

That is where most waste happens.

Typical agentic coding waste includes:

- repeated full-file reads;
- giant terminal logs sent upstream verbatim;
- persistent exposure of large tool schemas;
- repeated loading of repo structure or directory listings;
- duplicate error loops;
- risky tool execution without policy mediation; and
- overuse of expensive frontier models for low-complexity tasks.

BEAST tackles this by making local compute useful before cloud inference is invoked.

The key architectural insight

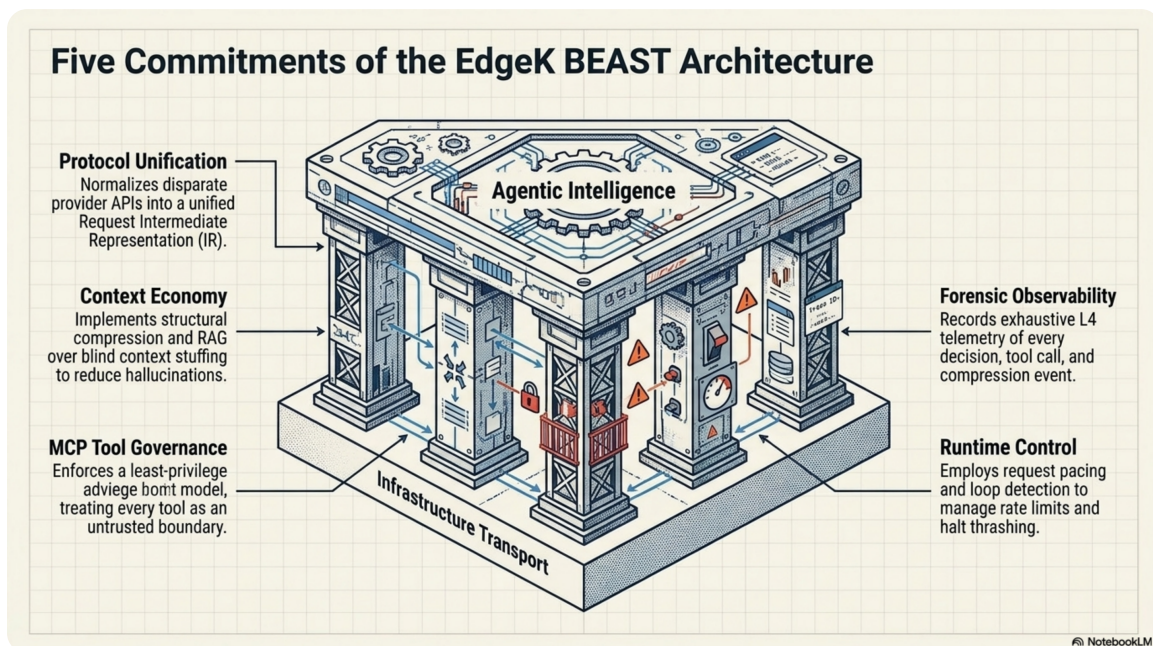
A small local model such as an Ollama-served model does **not** need to solve the entire coding task. It only needs to be smart enough to do low-cost first-pass cognition:

- classify the task;
- summarize files;
- suggest retrieval queries;
- rank retrieved context;
- redact sensitive content;
- build a structured handoff packet; and
- verify cloud output locally.

That simple shift transforms the whole economics of the system.

4. Architecture Overview

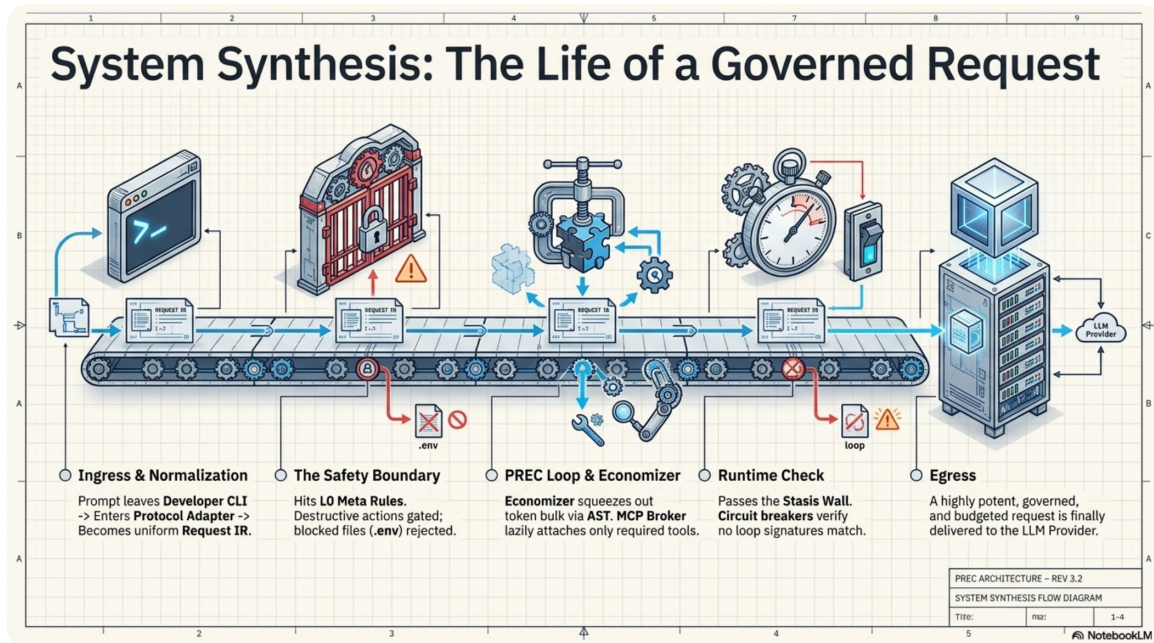
Five architectural commitments from the architecture deck



Five Commitments of the EdgeK BEAST Architecture

The architecture deck captures the system's philosophical center well: protocol unification, context economy, MCP tool governance, runtime control, and forensic observability.

High-level synthesis

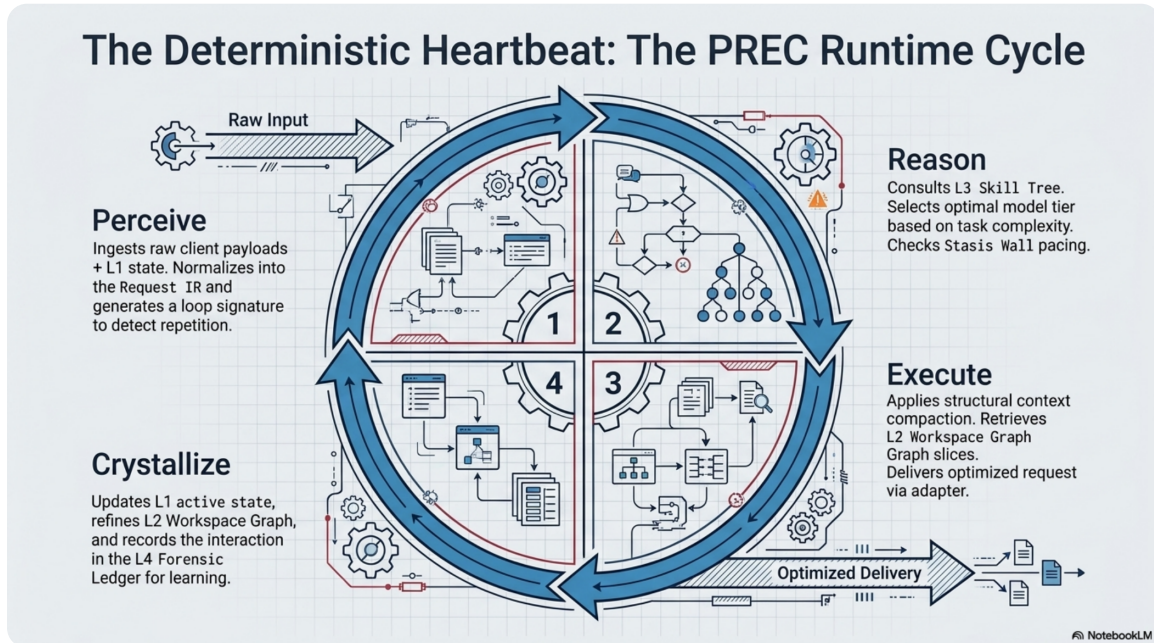


System Synthesis: The Life of a Governed Request

The strongest version of the system is not “many disconnected tools”. It is a **single request lifecycle** governed end to end:

1. receive request;
2. normalize it;
3. acquire and economize context;
4. apply policy and routing logic;
5. execute under runtime control;
6. validate and archive;
7. learn from successful traces.

The PREC cycle



The Deterministic Heartbeat: The PREC Runtime Cycle

The **PREC cycle** is one of the most coherent conceptual devices in the whole system:

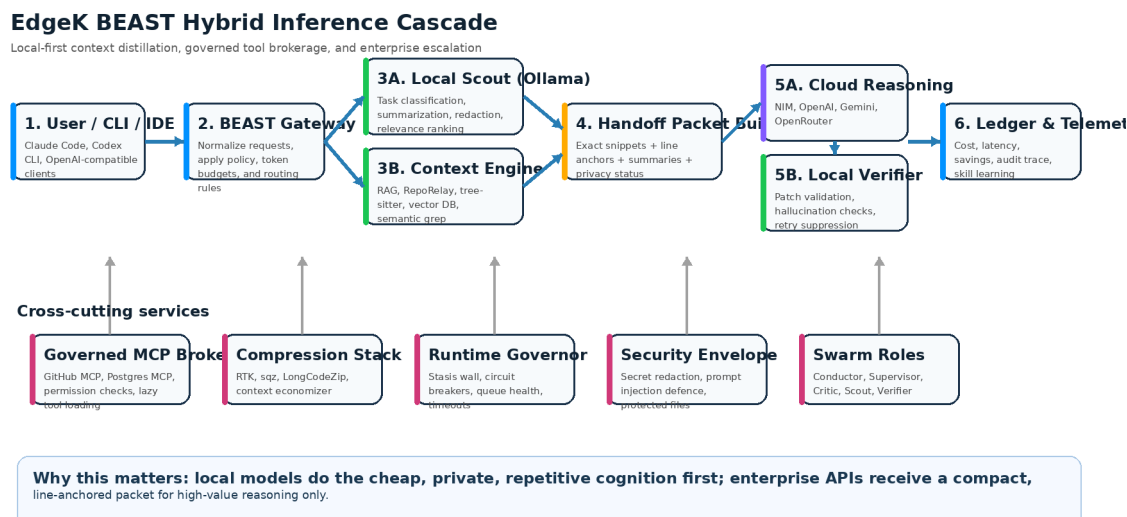
- **Perceive** – normalize the incoming request and ingest relevant environment state.
- **Reason** – evaluate policies, budgets, tool permissions, and routing rules.
- **Execute** – call the selected provider and/or tool path under runtime controls.
- **Crystallize** – archive the trace, emit telemetry, update workspace state, and learn reusable skill sequences.

That cycle gives BEAST a deterministic backbone.

5. The Local-First Hybrid Inference Cascade

One of the biggest advances in the evolved BEAST concept is the **local inference context transfer** or **hybrid inference cascade** model.

Generated flow chart



EdgeK BEAST Hybrid Inference Cascade

What this means in practice

Instead of immediately forwarding a coding request to a cloud model, BEAST can do the following first:

1. accept the request from a coding client;
2. normalize it into a provider-neutral request IR;
3. use local retrieval, graph, and parsing tools to collect candidate context;
4. use a local Ollama model to classify the task and rank what matters;
5. build a structured handoff packet containing exact snippets, line anchors, summaries, security status, and confidence;
6. escalate only the distilled packet to NIM, OpenAI, Gemini, or another enterprise API if the task warrants it;
7. locally verify the response before allowing tool execution or patch application;
8. record the full transaction in the telemetry and skill-learning layers.

Why this is powerful

This idea is genuinely strong because it uses **cheap local cognition to make expensive cloud cognition sharper**.

BEAST does not need local models to be perfect. They only need to be good at:

- scouting;
- summarizing;
- filtering;
- packing;
- verifying; and
- routing.

That makes the system much more commercially plausible than any attempt to compete by training a new frontier model.

6. Major Integrations and Their Roles

This section consolidates the integration ideas that emerged across the discussions.

6.1 Core gateway and routing stack

Integration	Role in BEAST	Notes
Nginx	ingress, TLS, buffering, reverse proxy, timeout handling	perimeter layer, not the intelligence layer
LiteLLM	provider abstraction and unified routing	suitable backbone for OpenAI, Anthropic, Gemini, NIM, OpenRouter, HF
FastAPI / local core	custom BEAST kernel	ideal for the governance, runtime, and trace logic

6.2 Local inference and hybrid compute

Integration	Role in BEAST	Notes
Ollama	local scout / summarizer / classifier / verifier	central to the local-first cascade
Jetson (especially Orin Nano)	edge sidecar for embeddings, reranking, tiny local inference, secret scanning	useful as a persistent local node
CPU-only local models	BEAST Lite local cognition	slower, but viable for summaries, routing, and retrieval support
NVIDIA NIM	heavy cloud inference for coding and reasoning	ideal “big model” escalation target

6.3 Tool governance and MCP ecosystem

Integration	Role in BEAST	Notes
GitHub MCP	repo browsing, issues, PRs, diffs	should be permissioned and scoped
Postgres MCP	schema inspection, read-only query workflows	default to read-only and bounded queries
MCP broker	central least-privilege vault	should hide unnecessary tool schemas until needed

6.4 Context economy and compression stack

Integration	Role in BEAST	Notes
RepoRelay	MCP-native code context engine / workspace graph	ideal for repo graphing and structured retrieval
tree-sitter	exact symbol, function, import, and AST extraction	deterministic parser layer
vector DB (ChromaDB / Qdrant / SQLite vec / pgvector)	semantic retrieval	useful for file and code chunk recall
RTK (Rust Token Killer)	CLI output compression	great for logs, test output, and command traces
sqz	transcript and repeated-content squeezing	session-level compaction
LongCodeZip	code-aware budget packing	useful after retrieval, not before

6.5 Security, sandboxing, and enterprise acceleration

Integration	Role in BEAST	Notes
NemoClaw	sandboxing, isolation, safer always-on agent operation	useful for the security kernel
OpenClaw	optional assistant / shell front end	UI and workflow shell, not the core governor
Dynamo	future self-hosted inference plane	more relevant once BEAST runs serious self-hosted models
Grove / DSX	future orchestration / AI-factory scale	relevant later for enterprise deployments

Summary judgment

The most immediately valuable integrations are:

- LiteLLM;
- Ollama;
- RepoRelay;
- tree-sitter;
- RTK / sqz / LongCodeZip;
- GitHub MCP;
- Postgres MCP;
- NIM;
- and a strong custom runtime/governance core.

7. Context Interception and Economization

The context interception idea is one of the best parts of the system.

The problem

If Claude Code or another agent is allowed to freely read files, dump logs, and enumerate tools before the gateway sees the content, the prompt is already bloated. In other words, post-hoc compression helps, but preemptive interception helps more.

The better design

BEAST should intercept tool intent and substitute **context-aware retrieval** for blunt raw reads whenever possible.

Instead of:

- `cat entire_file.py`
- `read full 3,000-line service`
- `dump every table and schema`
- `load every MCP tool definition`

BEAST should do:

- retrieve relevant functions and signatures;
- return exact line-anchored excerpts;
- compress logs into failure summaries;
- expose only the 3-5 tools most likely to matter for the current task;
- attach provenance, hashes, and retrieval reasons.

What a handoff packet should contain

A mature BEAST handoff packet should include fields such as:

- task type;
- risk level;
- privacy level;
- relevant files;
- exact line ranges;
- summaries;
- compressed logs;
- excluded contexts;
- confidence score;
- constraints for the cloud model.

That makes the cloud reasoning stage far more precise.

| 8. Runtime Governance and Swarm Control

BEAST becomes especially interesting when its governance layer is treated as a **runtime operating system for agentic workflows**.

Runtime controls

The mature runtime should include:

- **stasis-wall pacing** to smooth requests under provider rate limits;
- **circuit breakers** to halt failing provider paths temporarily;
- **queue health tracking**;
- **timeout budgets**;
- **retry suppression**;
- **loop detection**;
- **session attempt ledgers**; and
- **admission control** before requests are even sent upstream.

Swarm roles

The system can use “swarm roles” without becoming a wasteful multi-agent token bonfire.

Useful internal roles include:

- **Conductor** – chooses the next processing step;
- **Supervisor** – enforces budgets and permissions;
- **Critic** – challenges repetitive or low-quality paths;
- **Scout** – focuses on retrieval and initial local reasoning;
- **Verifier** – checks safety and quality before execution.

This is best implemented as a **state machine of specialized functions first**, and only secondarily as multiple model conversations.

| 9. User Interface / Control Plane Vision

The ideal BEAST UI is not decorative. It is a **control plane** for the whole governed AI execution system.



EdgeK BEAST Control Plane Dashboard

Core UI sections

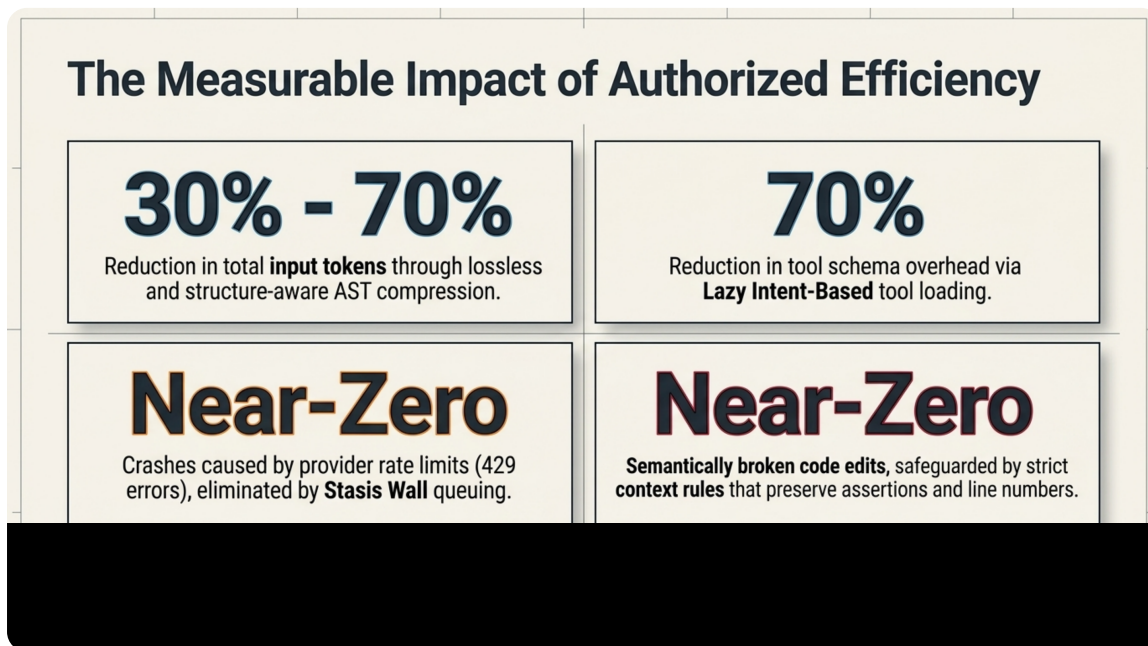
A strong UI should show:

- operational status and active sessions;
- total requests, savings, escalations, and interventions;
- local-versus-cloud workload distribution;
- token savings and compression ratios;
- context economy metrics such as intercepted file reads and semantic hits;
- security posture, prompt-injection alerts, and secret redactions;
- MCP and tool usage patterns;
- swarm role health;
- cost and provider-spend breakdown;
- queue health, latency, and cache hit rates;
- recent security and runtime events;
- and an activity log tracing real request flows.

In short: the UI should make BEAST feel like a **mission-control room for governed inference**.

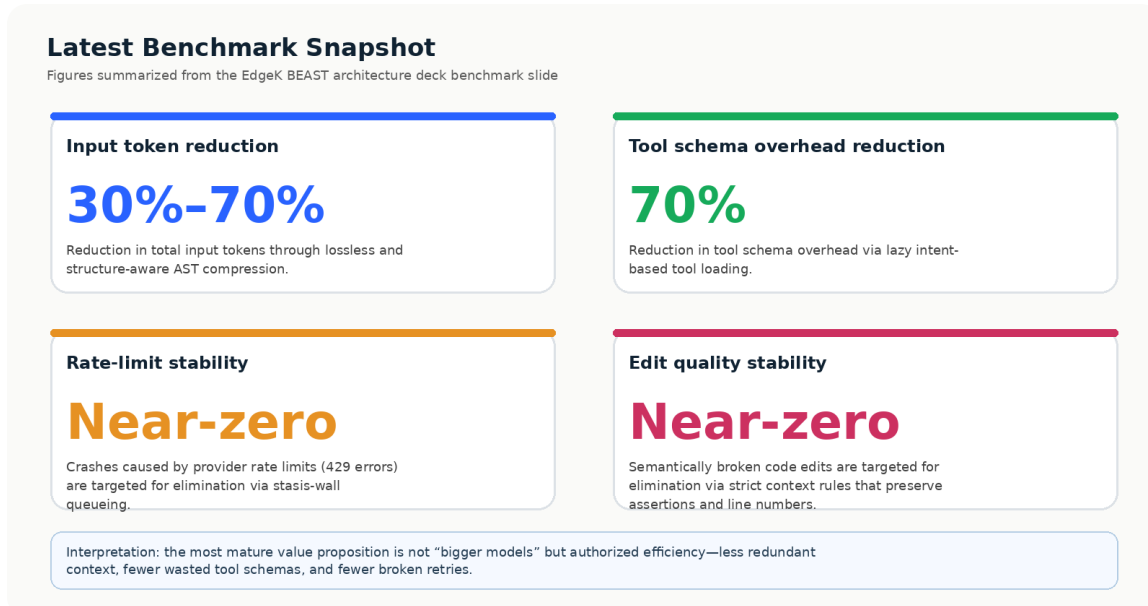
10. Latest Benchmark Review

Benchmark slide from the architecture deck



Benchmark slide from the architecture deck

Generated benchmark summary



Benchmark summary

Benchmark figures highlighted in the deck

The benchmark slide in the architecture PDF highlights the following outcomes:

- **30%-70% reduction in total input tokens** through lossless and structure-aware AST compression;

- **70% reduction in tool schema overhead** via lazy intent-based tool loading;
- **near-zero 429/rate-limit crashes** through stasis-wall queueing; and
- **near-zero semantically broken edits** through stricter context rules preserving assertions and line numbers.

Commentary on the benchmark

These are very strong and commercially meaningful figures **if validated in implementation**.

The most compelling thing about the benchmark is not just the raw savings figure. It is the pattern of what is improving:

1. **Input efficiency** – less junk goes upstream.
2. **Schema efficiency** – tool exposure is governed rather than permanent.
3. **Runtime stability** – fewer operational crashes from rate limiting.
4. **Edit stability** – less garbage-in / garbage-out behavior during coding edits.

Best interpretation

The benchmark suggests that BEAST’s most persuasive value proposition is **authorized efficiency**, not “bigger context” or “more AI for the sake of more AI”.

Important caution

These benchmark claims should be treated as **architecture-deck results or targets**, not yet as independently reproduced public benchmarks. For a production or investor-facing version, I would recommend generating a formal benchmark harness with:

- fixed repository sets;
- fixed prompt suites;
- multiple providers and local models;
- repeated-run statistics;
- patch-quality scoring;
- security event logging;
- and side-by-side comparisons with and without BEAST.

That would transform the strong story into even stronger evidence.

11. Patent-Supported Novelty Highlights

The uploaded provisional specification is helpful because it already frames the invention in a careful, non-overclaiming way.

Title in the provisional draft

Governed Agentic Execution Gateway with Normalized Request Representation, Context Economisation, Policy-Based Tool Brokerage, Runtime Circuit Breaking, and Trace-Derived Meta-Tool Promotion

Novelty themes that stand out

Based on the uploaded provisional specification, the novelty appears to lie not in any one third-party tool, protocol, or model provider, but in the **particular arrangement and processing sequence** that combines them.

The strongest novelty points are:

1. Normalized provider-neutral request representation

Incoming OpenAI-style, Anthropic-style, and other heterogeneous requests are converted into a shared internal representation before governance. That normal form is important because it allows every later layer to operate consistently.

2. Context economization as a governed stage

The invention does not merely compress text. It performs **policy-driven, request-aware context economization** while preserving protected content and recording the compression decision as part of the trace.

3. Policy-governed MCP and tool brokerage

The broker does not assume tools are trusted. It classifies, evaluates, approves, denies, audits, redacts, and conditionally executes narrowly supported operations under policy.

4. Runtime admission control

The runtime governor turns model execution into a governed transaction with:

- stasis-wall concurrency controls;
- circuit breaker state;
- attempt logs;
- retry metadata; and
- durable execution tracking.

5. Trace-derived meta-tool learning

This is especially interesting. Successful recurring tool and reasoning patterns are mined, validated, and promoted into reusable skills or meta-tools. That moves the system from static routing toward **learned operational refinement**.

6. Workspace graph and forensic archive

The trace and graph layer ties together sessions, traces, providers, policies, files, and economy events in a durable, queryable structure.

Why the novelty case is credible

A lot of people can claim “proxy + tools + RAG + metrics”. The stronger case here is the **integrated governance cycle**:

1. normalize;
2. economize;
3. reason under policy;
4. admit / queue / execute;
5. archive and learn.

That sequence is where the inventive coherence lies.

Suggested whitepaper phrasing

A safe and attractive way to describe the novelty without overclaiming is:

EdgeK BEAST’s novelty lies in the integrated combination of provider-neutral request normalization, deterministic context economization, policy-governed tool brokerage, runtime admission control, and trace-derived meta-tool learning within a single governed execution lifecycle.

Important note

This is an architecture and commercialization discussion, **not legal advice**. The provisional draft should still be reviewed by a patent attorney before any final filing strategy or public-claims language is locked in.

12. Commercial Relevance

The commercial relevance of BEAST is stronger than it may initially seem.

Who might want this

- engineering teams using AI coding assistants at scale;
- consultancies managing cost and quality across multiple client repos;
- universities and labs handling private code and sensitive research workflows;
- enterprises requiring auditability, policy control, and local-first safeguards;
- and platform teams building internal AI engineering environments.

Why they may care

BEAST offers a combination that is hard to get from a single off-the-shelf product:

- cost reduction;
- tool governance;
- local privacy controls;
- multi-provider flexibility;
- observability;
- resilience under rate limits;
- and a path to learned process automation.

The practical commercial story

The sales story is not “our model is smartest.”

The sales story is:

“We make your existing AI coding stack cheaper, safer, more observable, and more reliable.”

That is a very good story.

13. MVP, Deployment Tiers, and Roadmap

MVP recommendation

A sensible first version of BEAST should focus on one killer workflow:

Fix a coding issue using local context distillation, escalate only the distilled packet to a cloud model, then locally verify the patch.

MVP stack

- FastAPI-based BEAST kernel;
- LiteLLM for provider routing;
- Ollama for local scout / summarizer / verifier;
- tree-sitter for parsing;
- RepoRelay or a lightweight repo graph layer;
- vector retrieval using ChromaDB, pgvector, SQLite vec, or Qdrant;
- RTK/sqz/LongCodeZip-inspired compaction;
- GitHub MCP;
- optional Postgres MCP;

- NIM and at least one additional cloud provider for comparison;
- JSONL + SQLite trace archive;
- and a first-pass dashboard.

Deployment tiers

Tier 1 - BEAST Lite

- laptop or workstation;
- CPU or Ollama local model;
- cloud provider for hard reasoning;
- minimal dashboard and governance.

Tier 2 - BEAST Edge

- add Jetson or Jetson Orin sidecar;
- embeddings, reranking, security scanning, small local models;
- stronger always-on local services.

Tier 3 - BEAST Enterprise

- multiple providers;
- self-hosted inference options;
- richer MCP estate;
- central control plane;
- advanced telemetry and policy packs.

Tier 4 - BEAST Factory Scale

- Dynamo / Grove / DSX or equivalent orchestration;
- full AI-platform integration;
- multi-tenant internal developer environments.

Roadmap priorities

1. working gateway and request IR;
 2. context interception and local handoff packet builder;
 3. Ollama hybrid cascade;
 4. tool governance and runtime governor;
 5. telemetry and dashboard;
 6. trace mining and meta-tool promotion;
 7. enterprise hardening and benchmark harness.
-

| 14. Practical Risks and Design Boundaries

A good whitepaper should be honest about the boundaries.

1. Do not market as “quota bypassing”

BEAST should never be framed as stealthily defeating provider billing or server-side token accounting. That is the wrong story and the wrong risk posture.

2. Local models can over-compress

If the local model strips away the one exact detail a patch depends on, the downstream cloud model may fail. That is why line-anchored exact snippets and confidence metadata matter.

3. MCP is a security surface

Tool servers are not harmless. They are powerful execution surfaces. Least privilege, approval flow, allowlists, and auditability are non-negotiable.

4. Benchmarking must be formalized

The deck metrics are exciting, but production confidence requires a reproducible benchmark harness.

5. Swarm design must stay disciplined

If “swarm” turns into uncontrolled multi-agent chatter, the system will multiply cost instead of reducing it.

| 15. Conclusion

EdgeK BEAST is a genuinely strong systems idea.

Its power does not come from inventing a new LLM. It comes from making existing models, tools, and coding agents behave as part of a **governed execution workflow**.

The deepest strategic insight in the project is the local-first hybrid cascade:

- local tools and local models do the cheap, private, repetitive cognition first;
- the gateway packages a disciplined handoff;
- enterprise APIs do the high-value reasoning only when needed;
- local verification and telemetry close the loop;
- repeated successes become reusable skills.

That is why the concept has both technical elegance and commercial potential.

In one sentence:

EdgeK BEAST transforms AI coding from a token-hungry, tool-sprawling, error-prone workflow into a governed, measurable, and economically disciplined engineering system.

| Appendix A - Included Visual Assets

The following images are embedded in this Markdown document:

- `edgek_beast_assets/edgek_beast_logo.png`
- `edgek_beast_assets/arch_commitments-04.png`
- `edgek_beast_assets/arch_prec_cycle-10.png`
- `edgek_beast_assets/arch_system_synthesis-12.png`
- `edgek_beast_assets/arch_benchmark-15.png`
- `edgek_beast_assets/edgek_beast_hybrid_flow.png`
- `edgek_beast_assets/edgek_beast_dashboard.png`
- `edgek_beast_assets/edgek_beast_benchmark_summary.png`

| Appendix B - Source Inputs Used

This integrated Markdown document was synthesized using the following user-supplied materials:

- `EdgeK_BEAST_Architecture.pdf`
- `EdgeK_BEAST_CIPC_Provisional_Specification_Implementation_Supported(1).pdf`
- `edgek_beast_gateway_whitepaper(2).md`
- prior discussion outputs on BEAST, Ollama, Jetson, NIM, MCP, runtime governance, and UI/control-plane concepts.